

Abuse-Resistant OTP System

Assessment - SDE

Language	Python or Node.js
Auth lib	Not permitted - implement yourself

OVERVIEW

Build the backend for an OTP-based login flow. A user provides their email or phone number, receives a one-time code, and submits that code to authenticate. The core challenge is not the happy path - it is making the system resistant to three specific attacks: an attacker brute-forcing the 6-digit code by trying all combinations, a bot spamming the send-code endpoint to exhaust your SMS or email budget, and a stolen code being reused after a successful login. You will implement all security logic yourself without using any authentication library.

WHAT YOU NEED TO BUILD

1. Code generation and storage

When a user requests a code, generate a cryptographically random 6-digit OTP. Do not use `Math.random()` or the random module - use a cryptographically secure source (e.g. `secrets.token_bytes` in Python, `crypto.randomInt` in Node.js). Store the code hashed, not in plaintext. Include the identifier (email or phone), a `created_at` timestamp, an expiry, a used flag, and an attempt counter alongside the stored hash.

2. Rate limiting - [rate limiter](#) – [smudge.ai blog](#)

Apply two independent rate limits. First, limit how many times a given identifier can request a new code within a time window - this prevents SMS cost attacks. Second, limit how many times a given identifier can submit an incorrect code before the current OTP is invalidated - this prevents brute force. Both limits must be tracked per identifier, not per IP address. In-memory tracking is acceptable for this assessment.

3. Verification and invalidation

On a verify request, hash the submitted code and compare it to the stored hash. Reject the attempt if the code has expired, if the used flag is already true, or if the attempt limit has been reached. On a successful verification, immediately set used to true. The same code must not verify successfully a second time, even if submitted within the expiry window.

SECURITY RULES

All of the following must be enforced. The On violation column is the HTTP status your server must return.

Behaviour	Rule	On violation
OTP storage	Store a bcrypt or SHA-256 hash of the code. Never store the plaintext code.	500*
Code expiry	OTP expires 10 minutes after generation. Reject any verify attempt after expiry.	400
Brute-force limit	Lock the OTP after 5 incorrect verify attempts. Return an error; do not reveal the code.	429
Send rate limit	Max 3 send requests per identifier per 10-minute window.	429
Code reuse	A verified code must be invalidated immediately. A second verify with the same code fails.	400
CSPRNG	OTP must be generated using a cryptographically secure random source.	500*

* These are implementation requirements, not runtime errors. Violating them means the build fails the security criteria regardless of whether the endpoint returns 200.

API ENDPOINTS

Method	Endpoint	Request body	Response
POST	/auth/send	{ "identifier": string }	200 on success. 429 if rate limit exceeded.
POST	/auth/verify	{ "identifier": string, "code": string }	200 with token on success. 400 on expired/used/invalid. 429 on attempt limit.

On a successful verify, return a session token (any opaque random string is sufficient - full JWT implementation is not required). The shape of both responses must include a message field explaining the outcome.

```
// POST /auth/send - success
{ "message": "OTP sent to user@example.com" }

// POST /auth/verify - success
{ "message": "Verified", "session_token": "<random_opaque_string>" }

// POST /auth/verify - failure
{ "message": "Invalid or expired code", "attempts_remaining": 3 }
```

CONSTRAINTS

- No authentication libraries (Passport, NextAuth, Firebase Auth, Django Allauth, or equivalent). All logic must be your own code.
- Storage: use any database or in-memory store. SQLite and in-memory are both acceptable.
- You do not need to integrate a real SMS or email provider. Log the OTP to the console during development and document this in your README.
- Do not use Math.random() or Python's random module for OTP generation. Use a CSPRNG. State which function you used in your README.

DELIVERABLES

- Working /auth/send and /auth/verify endpoints, runnable from your README instructions.
- README covering: setup steps, your CSPRNG function choice, your hashing approach and why, and how the rate limits are tracked.
- A note in the README: if your OTP table were leaked, what could an attacker recover from it and what could they not?

EVALUATION

Criterion	Weight	What we look for
CSPRNG and hashed storage - code is never stored in plaintext	30%	Correct function used; hash is stored, not the raw code.
Brute-force protection - OTP locks after 5 failed attempts	25%	Attempt counter increments correctly; 429 returned on breach.
Send rate limiting - max 3 requests per identifier per 10 min	20%	Limit is per identifier; resets correctly after the window.
Code invalidation - used codes and expired codes are rejected	15%	used flag set on success; expiry enforced on every verify.
README security analysis - honest assessment of the threat model	10%	Candidate understands what the hash protects against and what it does not.

Instructions

Kindly create a private GitHub repository named: {yourName_Nestack_Submission} (e.g., roy_Nestack_Submission) and add the following contributors for evaluation:

- bishal@nestack.com
- sannidhya@nestack.com
- sanjay@nestack.com

Your submission must include:

- A ZIP file of your complete codebase
- A live deployment link
- Any necessary credentials if your project includes authentication

Additionally, ensure your repository includes a README.md with:

- The deployment link clearly mentioned
- Setup and run instructions if applicable

NOTE: PUBLIC REPOSITORIES ARE DISQUALIFIED FROM THE HIRING PROCESS